

Assignment 3 (v2)

Student Id : 202412048

Name : Harsh HemalKumar Mehta

Q1. Create a trigger on Table of your choice to check if the Primary key ID already exists or not , before inserting a new record. & Send a custom reply instead of an error message.

Ans)

Function:

```
CREATE OR REPLACE FUNCTION check_user_id()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $BODY$
BEGIN
    IF EXISTS (SELECT 1 FROM "EC_DB".users WHERE user_id = NEW.user_id)
    THEN
        RAISE EXCEPTION User ID already exists. Please choose a different ID.';
    END IF;
    RETURN NEW;
END;
$BODY$;
```

Trigger:

```
CREATE OR REPLACE TRIGGER trigger_q1
BEFORE INSERT ON "EC_DB".users
FOR EACH ROW
EXECUTE FUNCTION check_user_id();
```

TestCase:

```
insert into "EC_DB".users values  
(1,'tmp','tmp@gamil.com','asdfg','tmp','tmp','tmp');
```

Data Output	Messages	Notifications
ERROR: User ID already exists. Please choose a different ID. CONTEXT: PL/pgSQL function check_user_id() line 5 at RAISE SQL state: P0001		
Total rows: 100 of 100	Query complete 00:00:00.130	Ln 16, Col 9

Q2. Create a trigger on the Table of your choice to check if the Foreign key ID already exists or not before inserting a new record. & Send a custom reply instead of an error message.

Ans)

Function:

```
CREATE OR REPLACE FUNCTION check_fk_id()
```

```
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
```

```
AS $BODY$
```

```
BEGIN
```

```
    IF EXISTS (SELECT 1 FROM "EC_DB".orders WHERE user_id = NEW.user_id)  
    THEN
```

```
        RAISE EXCEPTION 'User ID foreign key already exists.';
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$BODY$;
```

Trigger:

```
CREATE OR REPLACE TRIGGER trigger_q2
BEFORE INSERT ON "EC_DB".orders
FOR EACH ROW
EXECUTE FUNCTION check_fk_id();
```

TestCase:

```
insert into "EC_DB".orders values (101,39,'24-07-
25','abcdefgh',2525.5,'Pending');
```

Data Output Messages Notifications

ERROR: User ID foreign key already exists.
CONTEXT: PL/pgSQL function check_fk_id() line 5 at RAISE

SQL state: P0001

Total rows: 100 of 100 Query complete 00:00:00.157

Ln 44, Col 17

Q3. Write a SQL function that takes a product's product id and a discount percentage as inputs and returns the discounted price Of the product.

Ans)

Function:

```
CREATE OR REPLACE FUNCTION get_disc_price(pro_id INT, disc_per NUMERIC)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $BODY$
DECLARE
    og_price NUMERIC;
    disc_price NUMERIC;
BEGIN
```

```
SELECT price INTO og_price  
FROM "EC_DB".products  
WHERE product_id = pro_id;
```

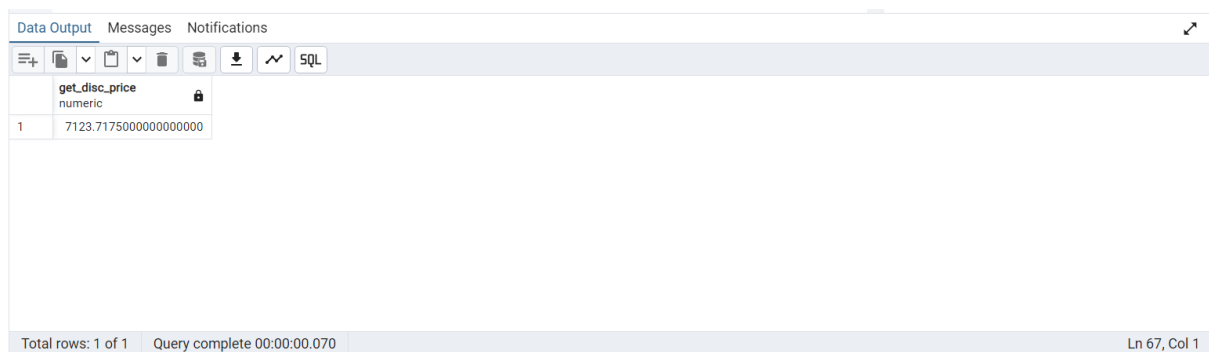
```
disc_price = og_price - (og_price * disc_per / 100);
```

```
RETURN disc_price;
```

```
END;
```

```
$BODY$;
```

TestCase: select get_disc_price(1,5);



get_disc_price numeric	
1	7123.717500000000000000

Total rows: 1 of 1 Query complete 00:00:00.070 Ln 67, Col 1

Q4. Create a stored procedure named add_order that takes the user_id, shipping_address, and a list of product_id and quantity pairs as inputs, and inserts a new order into the orders and order_details tables. The procedure should also update the stock quantity of the products.

Ans) Didn't understand how to do it

Q5. Write a trigger that automatically decreases the stock quantity of a product in the products table when a new order is inserted into the order details table.

Ans)

Function:

```
CREATE OR REPLACE FUNCTION dec_stock_qty()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $BODY$  
BEGIN  
  
    UPDATE "EC_DB".products  
    SET stock_quantity = stock_quantity - NEW.quantity  
    WHERE product_id = NEW.product_id;  
    RETURN NEW;  
END;  
$BODY$;
```

Trigger:

```
CREATE OR REPLACE TRIGGER trigger_q5  
AFTER INSERT ON "EC_DB".order_details  
FOR EACH ROW  
EXECUTE FUNCTION dec_stock_qty();
```

Test Case:

```
INSERT INTO "EC_DB".ORDER_DETAILS VALUES (302,100,50,10,6000)  
select * from "EC_DB".order_details;
```

Data Output

Messages

Notifications

(Quantity Reduced)